

Back-end Frameworks

WEBSERVICE: A importância das APIs no desenvolvimento backend

Profº: Joseph Donald

Contatos:

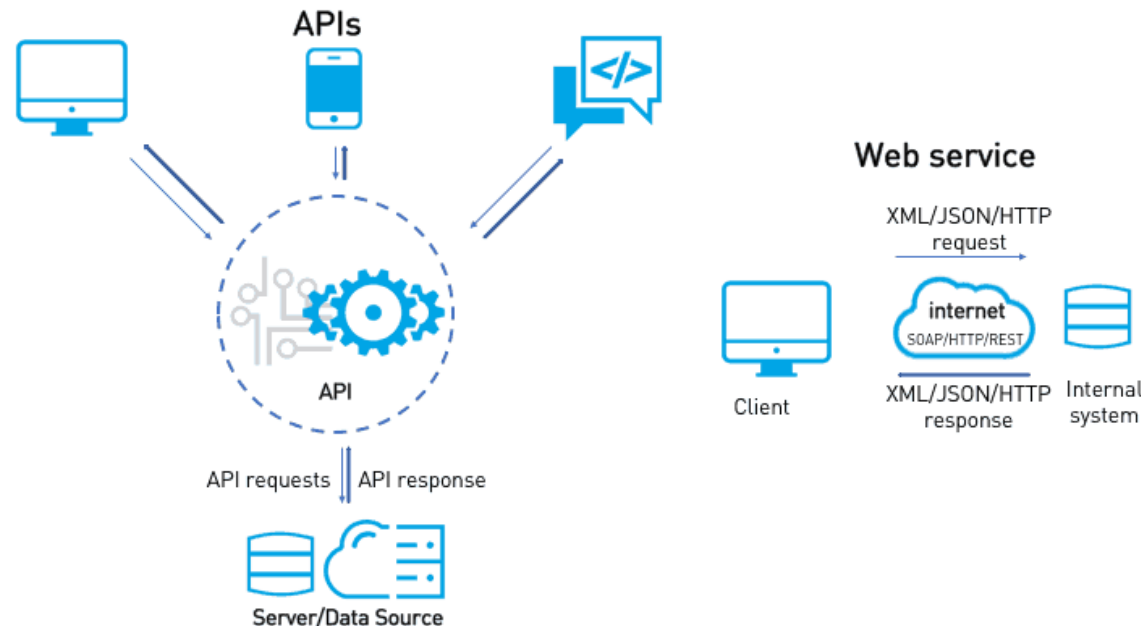
 www.josephdonald.wordpress.com

 @josephdonald

 030106382@prof.uninassau.edu.br

“O sucesso consiste em ir de fracasso em fracasso sem perder o entusiasmo.”
Winston Churchill

- ✓ Segundo Sommerville(2018): *“É uma representação padrão de algum recurso computacional ou informacional que pode ser utilizado por outros programas.”*
- ✓ Uma interface webservice fornece funcionalidades e/ou parâmetros de serviço, mas não requisita.





- Benefícios:
 - ✓ Os serviços podem ser fornecidos dentro e fora da organização.
 - ✓ O fornecedor torna públicas as informações sobre o serviço, para que qualquer usuário autorizado possa usá-lo.
 - ✓ Aplicações podem ser reativas e adaptar sua operação para lidar com mudanças no seu ambiente de execução.
 - ✓ As aplicações podem ser menores, fato que auxilia muito na capacidade de processamento de dispositivos mais limitados, como Smartphones, Smart TVs, Smartwatch e etc.



- API é a sigla em inglês para Application Programming Interface, ou interface de programação de aplicações.
- APIs conectam soluções e serviços, sem a necessidade de saber como esses elementos foram implementados.
- As APIs simplificam a forma como os desenvolvedores integram novos componentes de aplicações a uma arquitetura preexistente.



- As APIs geralmente utilizam formatos de dados padronizados para representar e transmitir informações. Um formato comum é o JSON (JavaScript Object Notation), que é leve, legível por humanos e amplamente suportado. Além do JSON, outros formatos como XML (eXtensible Markup Language) também podem ser usados em determinados contextos.
- O JSON trabalha com o padrão de “chave”:“valor”. Exemplo: “animal”:“Bob”
- O XML trabalha com TAGs. Exemplo: <animal>Bob</animal>

JSON

```
{  
  "user": {  
    "id": "11",  
    "animal": "Bob",  
    "idade": "2"  
  }  
}
```

XML

```
<user>  
  <id>11</id>  
  <animal>Bob</animal>  
  <idade>2</idade>  
</user>
```



- Em Python, o **jsonify()** é uma função fornecida pela biblioteca Flask.
- Essa função é comumente usada para converter objetos Python em formato JSON, tornando-os adequados para uso em APIs ou para enviar como resposta a solicitações da web.

Exemplo de consulta dos dados entregues em JSON

```
7 @app.route('/')
8 def index():
9     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
10    cursor = banco.cursor()
11    sql = "SELECT * FROM funcionario;"
12    cursor.execute(sql)
13
14    resultadoFetchAll = cursor.fetchall()
15    banco.close()
16
17    listaFuncJSON = []
18
19    for funcionario in resultadoFetchAll:
20        listaFuncJSON.append({
21            "nomeFuncionario": funcionario[1]
22        })
23
24    return jsonify(listaFuncJSON)
```

Resultado
JSON
acessando
a rota

```
1 // 20240509082103
2 // http://127.0.0.1:5000/
3
4 [
5   {
6     "nomeFuncionario": "Joseph Donald"
7   },
8   {
9     "nomeFuncionario": "Renária Maria"
10  },
11  {
12    "nomeFuncionario": "Lara"
13  },
14  {
15    "nomeFuncionario": "Livia"
16  },
17  {
18    "nomeFuncionario": "Napoleão"
19  },
20  {
21    "nomeFuncionario": "Barnabe"
22  },
23  {
24    "nomeFuncionario": "Antonio"
25  }
26 ]
```



CREATE

- Observe que na linha 29 o dado está sendo recebido através da função `get_json()`, e na linha 31 o conteúdo é recebido através do termo `dadosRecebidos["nomeFuncionario"]`. "nomeFuncionario" é a **chave** que veio através da API JSON.

Exemplo:

POST localhost:5000/cadastraFuncionario

Params Authorization Headers (10) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 {"nomeFuncionario": "Bernardo"}

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "codigo": 200,
3   "mensagem": "Cadastrado realizado."
4 }
```

```
27 @app.route(rule: '/cadastraFuncionario', methods=['POST'])
28 def cadastraFuncionario():
29     dadosRecebidos = request.get_json()
30
31     nome = str(dadosRecebidos['nomeFuncionario'])
32
33     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
34     cursor = banco.cursor()
35
36     sql = f"INSERT INTO funcionario (nome) VALUES ('{nome}');"
37
38     cursor.execute(sql)
39     banco.commit()
40     banco.close()
41
42     response = {'mensagem': 'Cadastrado realizado.', 'codigo': 200}
43
44     return jsonify(response)
```



READ

- Observe que entre as linhas 19 e 22 temos um FOR onde cada elemento do resultado do SELECT vai sendo adicionado numa lista utilizando o padrão JSON “chave”:“valor” e depois é retornado para a exibição no HTML.

Exemplo:

GET localhost:5000/

Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON

```
[
  {
    "nomeFuncionario": "Joseph Donald"
  },
  {
    "nomeFuncionario": "Renária Maria"
  },
  {
    "nomeFuncionario": "Lara"
  },
  {
    "nomeFuncionario": "Livia Maria"
  },
  {
    "nomeFuncionario": "Napoleão"
  },
  {
    "nomeFuncionario": "Antonio"
  },
  {
    "nomeFuncionario": "Bernardo"
  }
]
```

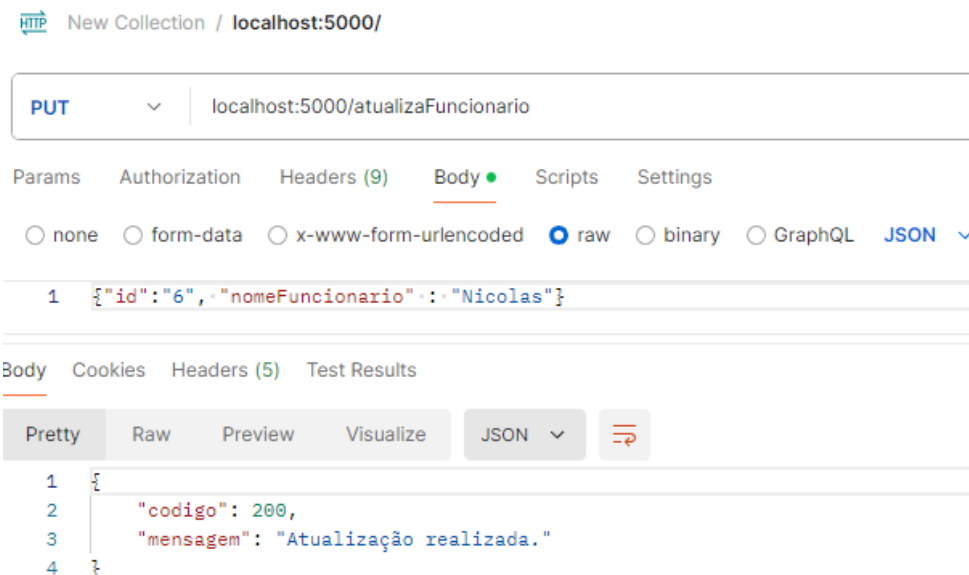
```
7 @app.route('/')
8 def index():
9     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
10    cursor = banco.cursor()
11    sql = "SELECT * FROM funcionario;"
12    cursor.execute(sql)
13
14    resultadoFetchAll = cursor.fetchall()
15    banco.close()
16
17    listaFuncJSON = []
18
19    for funcionario in resultadoFetchAll:
20        listaFuncJSON.append({
21            "nomeFuncionario": funcionario[1]
22        })
23
24    return jsonify(listaFuncJSON)
```




UPDATE

- Observe que na linha 47 o método utilizado é o PUT.
- Observe que na linha 50 o dado está sendo recebido através da função **get_json()**.
- Na linha 52 o conteúdo é recebido através do termo `dadosRecebidos['id']`. 'id' é a **chave** que veio através da API JSON.
- Na linha 53 o conteúdo é recebido através do termo `dadosRecebidos['nomeFuncionario']`. 'nomeFuncionario' é a **chave** que veio através da API JSON.

Exemplo:



```
47 @app.route(rule: '/atualizaFuncionario', methods=['PUT'])
48 def atualizaFuncionario():
49
50     dadosRecebidos = request.get_json()
51
52     idRecebido = dadosRecebidos['id']
53     nomeRecebido = dadosRecebidos['nomeFuncionario']
54
55     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
56     cursor = banco.cursor()
57
58     sql = f"UPDATE funcionario SET nome = '{nomeRecebido}' WHERE id_func = {idRecebido};"
59
60     cursor.execute(sql)
61     banco.commit()
62     banco.close()
63
64     response = {'mensagem': "Atualização realizada.", 'codigo': 200}
65
66     return jsonify(response)
```



DELETE

- Observe que na linha 69 o método passa a ser 'DELETE'.
- Na linha 72 o dado está sendo recebido através da função `get_json()`, e na linha 74 o conteúdo é recebido através do termo `dadosRecebidos['id']`. 'id' é a **chave** que veio através da API JSON.

Exemplo:

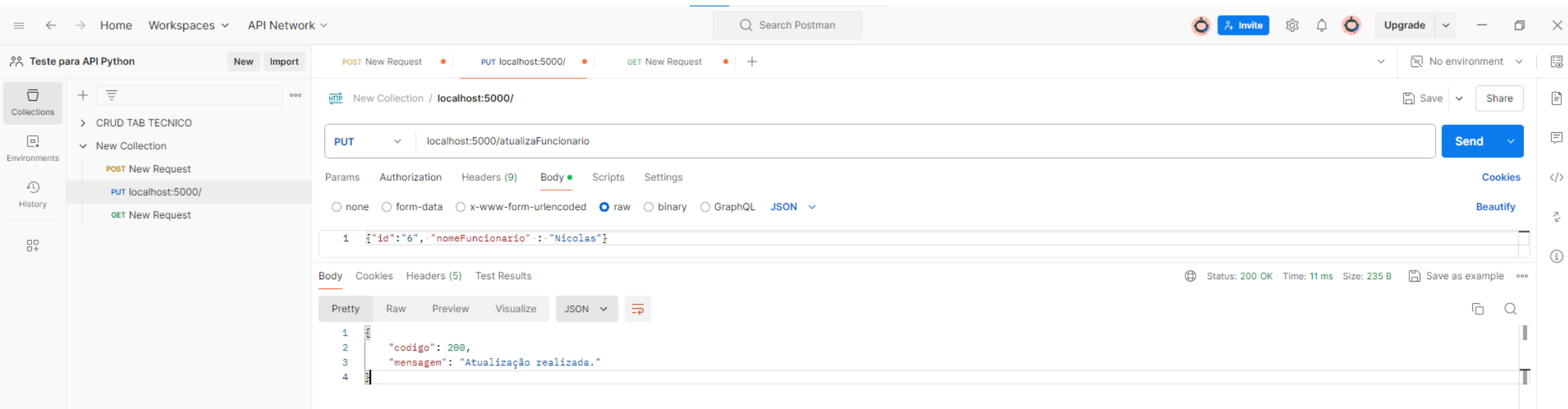
The screenshot shows a REST client interface. The top bar displays the method 'DELETE' and the URL 'localhost:5000/removerFuncionario'. Below this, there are tabs for 'Params', 'Authorization', 'Headers (9)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, and the 'raw' radio button is chosen. The body content is a JSON object: `{"id": "8"}`. At the bottom, there are tabs for 'Body', 'Cookies', 'Headers (5)', and 'Test Results'. The 'Body' tab is selected, and the 'JSON' format is chosen. The response is displayed as follows:

```
1 {  
2   "codigo": 200,  
3   "mensagem": "Deletado com sucesso."  
4 }
```

```
69 @app.route(rule='/removerFuncionario', methods=['DELETE'])  
70 def removeFuncionario():  
71  
72     dadosRecebidos = request.get_json()  
73  
74     idRecebido = dadosRecebidos['id']  
75  
76     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')  
77     cursor = banco.cursor()  
78  
79     sql = f"DELETE FROM funcionario WHERE id_func = {idRecebido};"  
80  
81     cursor.execute(sql)  
82     banco.commit()  
83     banco.close()  
84  
85     response = {'mensagem': 'Deletado com sucesso.', 'codigo': 200}  
86  
87     return jsonify(response)
```



- O POSTMAN é um software que auxilia na construção das APIs, pois nele você pode fazer simulação de requisições para o backend.
- Este é o link para download: <https://www.postman.com/downloads/>





Contextualização:

- Pateta vai montar uma lanchonete e precisa de um sistema que possa cadastrar o nome do produto, o preço, a quantidade e a data e hora do pedido através de um **aplicativo mobile feito em Flutter**.
- Baseando-se nas práticas que realizamos nas aulas anteriores e nos códigos disponíveis no Teams da disciplina, vamos ajudar o pateta desenvolvendo um software de cadastros. Para isto:

Requisitos do desafio:

1. Elabore um backend utilizando Flask que acesse o banco de dados e forneça uma API JSON realizando o CRUD dos seguintes campos:
 - a. ID
 - b. Nome do Produto
 - c. Preço
 - d. Quantidade
 - e. Data e Hora do Pedido
2. Os testes deverão ser realizados através do software POSTMAN.

IMPORTANTE: Na disciplina de Desenvolvimento para Dispositivos Móveis será desenvolvido o APP que vai acessar esta API.